ASSIGNMENT & EXAMINATION PAPER

Topic Set 1 — Group 1: Linked Lists (§4.1)

Total Points: 10.0 Points | Submission Window: Minimum 1 Week | Channel: LMS

Section 1 — Recall (1.0 point)

Problem Statement:

Complete the table below. For each of the specified linked list operations across different linked list variations (Singly Linked List, Doubly Linked List, Circular Linked List, and Sorted Linked List), fill in the Best, Average, and Worst-case time complexities, auxiliary space complexity, and — most importantly — the "Why" column: provide a brief theoretical justification explaining why each structure achieves that complexity.

Note: Filling in Big-O values without the "Why" explanation earns 0 points for this section.

Linked List Variation & Operation	Best	Average	Worst	Space	Why (Theoretical Justification)
Singly Linked List (Insert Head / Delete Tail)	?	?	?	?	?
Doubly Linked List (Delete Given Node Pointer)	?	?	?	?	?
Circular Linked List (Insert Head with TailPtr)	?	?	?	?	?
Sorted Singly Linked List (Search & Insert)	?	?	?	?	?

Grading Criteria:

- Correct Big-O values (Best/Average/Worst/Space) for all 4 list operations. (0.5 points)
- "Why" column: valid theoretical justification for all 4 operations. (0.5 points)

Section 2 — Implementation: Sorted Singly Linked List (2.5 points)

Problem Statement:

Implement a Sorted Singly Linked List algorithm from scratch in C++ to maintain an integer sequence in ascending order for an array of at least 100,000 elements.

Requirements:

- Do not use std::list, std::vector sorting, or any STL container or sorting function.

- Clearly separate the node insertion logic (insertSorted) and traversal/deallocation logic into distinct functions.
- Use the timeIt template below to measure the best runtime over 5 runs.
- Handle edge cases: empty list, single element, all duplicate elements, inserting at head, inserting at tail.
- Must compile cleanly with: g++ -std=c++17 -Wall -Wextra

How to measure by using the timeIt template:

```
#include <iostream>
#include <chrono>
#include <algorithm>

using namespace std;

template<typename F>
double timeIt(F f) {
    double best = 1e18;
    for (int i = 0; i < 5; i++) {
        auto t0 = chrono::high_resolution_clock::now();
        f();
        auto t1 = chrono::high_resolution_clock::now();
        best = min(best, chrono::duration<double, milli>(t1 - t0).count());
    }
    return best; // Return the fastest execution time in milliseconds out of 5 runs
}
```

Input:

- Line 1: integer n (0 ≤ n ≤ 1,000,000)
- Line 2: n space-separated integers (may contain duplicates)

Output:

- Line 1: sorted list elements in ascending order
- Line 2: SortedLinkedList: X ms

Sample:

Input:
6
5 3 8 1 3 2

Output:
1 2 3 3 5 8
SortedLinkedList: X ms

Grading Criteria:

- Program compiles cleanly, produces correct sorted output. (1.0 point)
- Clear separation of insertSorted function and helper/memory cleanup functions. (0.75 points)

- timeIt integrated correctly; output includes runtime. (0.5 points)
- Edge cases handled (empty list, duplicates, n=1). (0.25 points)

Section 3 — Implementation: Find the Middle Node of a Singly Linked List (2.5 points)

Problem Statement:

Write a C++ implementation to find the middle node of a Singly Linked List using the Two Pointers technique (Fast and Slow Pointers). Then, briefly explain the time complexity and space complexity of your algorithm.

Requirements:

- Implement a two-pointer mechanism: Start with both pointers at the head of the list. The slow pointer (slow) advances 1 step, while the fast pointer (fast) advances 2 steps per iteration.
- Determine the stopping condition: When the fast pointer reaches the end of the list (nullptr) or its next element reaches nullptr, the loop terminates. At this exact moment, the slow pointer will be at the middle node.
- Complexity Explanation: Write a short comment explaining why this algorithm has a time complexity of O(N) and an auxiliary space complexity of O(1).
- Handle edge cases:
    - Empty list (nullptr).
    - List with a single element.
    - List with an even number of elements (e.g., for 1 -> 2 -> 3 -> 4, it must return node 3).

Sample expected explanation (no lengthy proofs required):

Time Complexity: O(N)

Explanation: The fast pointer traverses the list at twice the speed of the slow pointer. To cover N elements, the loop only runs N/2 times. Dropping the constant factor, the time complexity is linear, O(N).

Auxiliary Space Complexity: O(1)

Explanation: The algorithm only uses two additional pointer variables (slow and fast). It does not allocate any new arrays or auxiliary data structures, thus requiring constant space.

Grading Criteria:

- 1.5 points: Correct implementation of the fast/slow pointer mechanism that accurately returns the middle node.
- 0.5 points: Proper handling of all edge cases (empty list, single element, even number of nodes).
- 0.5 points: Brief and accurate explanation for why Time = O(N) and Space = O(1).

Section 4 — Analysis: Singly vs. Doubly Linked List Traversal Counter (2.0 points)

Problem Statement:

Implement linear search on both a Singly Linked List and a Doubly Linked List with an embedded counter variable that counts the exact number of node pointer traversals performed during element lookups. Run it on the sample dataset provided by the instructor (n = 10,000 random integers).

Then construct a comparison table contrasting your empirical result against the theoretical bounds, and provide a written analysis explaining any discrepancy or advantage observed between the two structures.

Requirements:

- Place the counter increment at the exact location of node traversal (current = current->next or current = current->prev).
- For a list of size n = 10,000, theoretical worst-case search traversal is n = 10,000, and average-case for uniform random search is (n + 1) / 2 ≈ 5,000. For bidirectional search in Doubly Linked List (searching from closest end), theoretical average is ≈ n / 4 = 2,500.
- Fill in and submit the table below with your measured values:

n	Structure / Strategy	Empirical Traversals	Theoretical Worst	Theoretical Avg	Ratio (Empirical / Avg)
10,000	Singly Linked List (Head Search)	?	10,000	5,000	?
10,000	Doubly Linked List (Head Search)	?	10,000	5,000	?
10,000	Doubly Linked List (Bi-directional)	?	10,000	2,500	?

Written analysis:

Explain why bidirectional traversal in a Doubly Linked List reduces average search comparisons from n/2 down to n/4 when index or target proximity to head/tail is known, whereas Singly Linked List is strictly constrained to forward traversal.

Grading Criteria:

- Counter placed at exact location of node traversal in search loop. (1.0 point)

- Comparison table filled correctly with empirical data. (0.5 points)
- Written analysis explaining theoretical vs empirical findings (empirical ≈ n/2 for Singly vs ≈ n/4 for Doubly). (0.5 points)

Section 5 — Open/Design: Music Player Playlist System (2.0 points)

Problem Statement:

You need to design a backend data structure for a Music Player Playlist system that supports smooth track navigation (Next, Previous) and continuous looping (Repeat All mode).

Propose a linked list architecture to solve this problem. Explain its working principle in detail, covering track navigation and seamless looping workflows.

Your answer must address:

- The name and theoretical justification of the proposed data structure (Circular Doubly Linked List). Explain why Singly Linked List or standard Doubly Linked List is insufficient for seamless backward navigation and infinite looping.
- Phase 1: Detailed workflow for track navigation — how nextSong() and previousSong() operations work in O(1) time complexity by updating active pointer references without re-indexing.
- Phase 2: Detailed workflow for playlist management — how adding a track at any position, deleting a track, and infinite Repeat All looping work (leveraging head->prev = tail and tail->next = head links).

Grading Criteria:

- Correctly name and justify the proposed structure (Circular Doubly Linked List). (1.0 point)
- Accurate explanation of Phase 1 (O(1) next/previous track navigation). (0.5 points)
- Accurate explanation of Phase 2 (seamless looping & playlist modifications). (0.5 points)